Clearline

From a stablecoin transfer to a settled invoice

Aashmiin Kaur, founder

Working paper, version 0.1, 29 October 2026

Status, stated plainly. Clearline is a working product in demo mode, built by one founder. It has **no customers** and no customer conversations yet; outreach began on 29 October 2026. Live mode checks real Ethereum transactions when someone gives it the transaction ID. The Xero connection is built and tested against recorded API shapes but has not yet run on a live Xero organisation. Every number in this paper has a source, and every product claim maps to a file or test (see the claims audit that accompanies it).
Review copy. Every important claim carries a label: [VERIFIED] with its source (a footnote, or the file or test in the repository that proves it), [ASSUMPTION] for a hypothesis we have not yet tested, and [GUESS] for an estimate with no evidence behind it.

1. Abstract

Stablecoins now move real money between businesses. Bessemer estimates about US$400 billion of real-economy stablecoin payments in 2025, most of them business-to-business. [VERIFIED][1] Yet a transfer on a blockchain is not a completed business payment. A transfer says that a token balance changed. A payment means the right amount, of the genuine asset, on the agreed network, for a specific invoice, recorded correctly in the books. Between those two sits a layer that nobody clearly owns: telling the payer exactly what to send, checking what arrived, matching it to the invoice, explaining what went wrong, and posting the result to the accounting system.

Clearline is that layer for businesses paid directly into their own wallets. It issues a precise payment request for each invoice, checks each incoming transfer with deterministic rules, matches it to its invoice with evidence, explains failures in plain English, and prepares the accounting entries, which post only after a person approves. It is read-only on-chain and never holds funds. This paper describes the problem, the design, the evidence so far, what is not built, and the risks. It also proposes an open format for payment instructions and receipt evidence that wallets and accounting tools could share.

2. The problem: a transfer is not a payment

2.1 Two different things

A blockchain records **transfers**: token *X* moved from address *A* to address *B* in block *n*. A business needs **payments**: customer *C* settled invoice *I* in full, on time, in money the business can use. The second needs facts the chain does not hold:

- **Which invoice?** A USDC or USDT transfer on Ethereum carries no invoice reference. The transfer itself has no field for one.
- **Is it the genuine asset?** Anyone can deploy a token contract named "USDT". Only the issuer's contract makes it real.
- **Is it complete?** A transfer can be short (an exchange took its fee out), over, or sent twice.

[1] Bessemer Venture Partners, C. Birnbaum, E. Kaplan and B. Nydick, "Stablecoins: from DeFi primitive to global financial infrastructure", 22 April 2026. https://www.bvp.com/atlas/stablecoins-from-defi-primitive-to-global-financial-infrastructure

- **Did it arrive where we can use it?** The same address format exists on several networks. A transfer sent on the wrong one may never reach the business.
- **Is it final, and is it ours?** It needs enough confirmations, from a sender who is not frozen by the issuer, to an address the business actually controls.

A finance team must answer all five before it can close the invoice. Today that work is done by hand, and it is done after the money has already moved, when mistakes are hardest to fix.

2.2 How payments go wrong

Failure	What the chain shows	Why it is hard to see	Consequence
Short payment	4,950 USDC arrives for a 5,000 invoice	The amount looks "about right"	Invoice left open, or closed wrongly
Wrong network	USDT sent on Tron to an address issued for Ethereum	The payer's wallet says "sent"	Money may be unrecoverable, or sits in a wallet nobody checks
Counterfeit token	A token named "USDT" from a contract Tether does not control	The name and amount look correct	Worthless tokens credited as payment
Missing reference	2,180 USDC to a general address with no memo	Nothing links it to an invoice	Unapplied cash; the customer is chased for money they paid
Look-alike address	Funds sent to an address that shares the first and last characters with the real one	People check only the ends of a 42-character address	Funds go to a scammer before anyone looks
Duplicate	The same amount arrives twice, minutes apart	Each transfer is valid on its own	Over-collection, refund owed, books overstated

These are not hypothetical. On 19 December 2025, a user sent 49,999,950 USDT to an address that matched the intended one in its first three and last four characters, minutes after a small test payment to the right address. [VERIFIED][2] The transaction is public on Etherscan. [VERIFIED][3] In Clearline's own test of real public transactions (section 8), 13 counterfeit tokens calling themselves USDT appeared among the cases collected, one spelled with Cyrillic letters ("ÚSDT") to pass a visual check. [VERIFIED: demo/REPORT.md] That count reflects how the test cases were chosen, not how common counterfeits are.

2.3 How finance teams handle it today

We have not yet measured this with any customer. Our working picture, which the first customer calls are designed to test, is this: [ASSUMPTION] a customer emails "we paid yesterday, why is the invoice still open?"; someone opens a block explorer, finds the transfer, works out which invoice it was meant for, checks the token contract and the network, and then updates the invoice in the accounting system by hand, with a note copied from the explorer. Each exception takes a person's time and judgement. We guess 10 to 30 minutes per exception; [GUESS] the pilots will measure it.

The larger cost is risk, not time. A counterfeit token credited as payment is a loss. A look-alike address caught after the transfer is usually a loss. A wrong-network payment may be recoverable only if the business happens to control the same key on that network. And an auditor will ask how each stablecoin receipt was verified; a spreadsheet of explorer links is weak evidence.

[2] Cointelegraph, "Address poisoning: copy-paste mistake leads to $50M USDT loss", 20 December 2025. https://cointelegraph.com/news/address-poisoning-copy-paste-mistake-50m-usdt-loss

[3] Etherscan, transaction 0xc0514a79…6a0f. https://etherscan.io/tx/0xc0514a795f065fce8a3e1238a1ba480e8a792d45f6431ee4ba8d800bedd86a0f

3. Market context

3.1 Supply and real payments

Stablecoins are large and growing. The supply of fiat-backed stablecoins exceeded US$273 billion in March 2026, [VERIFIED][1] and total stablecoin market capitalisation was US$308.0 billion on 13 August 2026, according to DefiLlama data reported by Reap. [VERIFIED][4] Tether's USDT holds about 59% of supply and Circle's USDC about 23%. By network, Ethereum carries about 49% of stablecoin supply and Tron about 31%. [VERIFIED][4]

Payment use is real but small next to total activity. Bessemer reports that real-world stablecoin payment volume doubled in 2025 to about US$400 billion, of which an estimated 60% was business-to-business. [VERIFIED][1] Reap, citing the BIS, BCG with Allium, and McKinsey, puts genuine real-economy payments at about US$350–550 billion out of US$28–62 trillion of stablecoin transfers in 2025. [VERIFIED][4] On those figures, **real payments are roughly 0.6% to 2% of transfer volume**; the rest is trading and moving funds between wallets and exchanges. Any claim about "stablecoin volume" should be read with that ratio in mind. Clearline's market is the small slice that is real commerce.

3.2 What businesses use stablecoins for

In a mid-2025 EY-Parthenon survey of 350 corporate and financial-institution decision-makers, reported by Reap, the leading use cases were paying suppliers cross-border (62%) and accepting cross-border payments (53%). [VERIFIED][4] Reap describes these figures as a directional snapshot taken before the GENIUS Act was signed; we did not open the original EY-Parthenon report.

3.3 Regulation

Rules are now being written for issuers and intermediaries in all four markets most relevant to Clearline's first customers.

Jurisdiction	Status (checked 29 October 2026)
United States	The GENIUS Act became Public Law 119-27 on 18 July 2025. [VERIFIED][5] It takes effect on 18 January 2027. On 21 October 2026 the OCC finalised its implementing rule for issuers under its supervision; Treasury's anti-money-laundering and sanctions rules are a separate rulemaking. [VERIFIED][6]
European Union	MiCA's rules for e-money tokens and asset-referenced tokens have applied since 30 June 2024. [VERIFIED][7] The transitional period for crypto-asset service providers ended on 1 July 2026. [VERIFIED][8]
Hong Kong	The Stablecoins Ordinance came into operation on 1 August 2025. [VERIFIED][9] The HKMA granted the first issuer licences, to HSBC and Anchorpoint Financial, on 10 April 2026. [VERIFIED][10]

[4] Reap, "Stablecoin Statistics & Data 2026: All You Need To Know", 14 August 2026, citing BIS, BCG × Allium, McKinsey, DefiLlama and EY-Parthenon. https://reap.global/blog/stablecoin-statistics-2026

[5] U.S. Government Publishing Office, Public Law 119-27, "An act to provide for the regulation of payment stablecoins", 18 July 2025. https://www.govinfo.gov/metadata/pkg/PLAW-119publ27/mods.xml

[6] Decrypt, "OCC finalizes GENIUS Act stablecoin rules ahead of January deadline", 21 October 2026. https://decrypt.co/381502/occ-finalizes-genius-act-stablecoin-rules-ahead-of-january-deadline

[7] European Banking Authority, "Asset-referenced and e-money tokens (MiCA)". https://eba.europa.eu/regulation-and-policy/asset-referenced-and-e-money-tokens-mica

[8] FSMA (Belgium), "Crypto-asset service providers: end of the transitional period", 25 June 2026. https://www.fsma.be/en/news/crypto-assets-service-providers-end-transitional-period

[9] Hong Kong Government News, "Stablecoins laws effective Aug 1", 6 June 2025. https://www.news.gov.hk/eng/2025/06/20250606/20250606_140750_137.html

[10] Charltons, "HKMA Issues First Stablecoin Issuer Licences", April 2026. https://www.charltonslaw.com/hkma-issues-first-stablecoin-issuer-licences/

Jurisdiction	Status (checked 29 October 2026)
Singapore	On 1 September 2026, MAS consulted on draft amendments to the Payment Services Act for single-currency stablecoins, including a stand-alone issuance licence. Comments closed on 16 October 2026; no date has been set for enactment. [VERIFIED][11]

This regulation targets issuers and the firms that hold or transmit stablecoins. None of it closes the gap in section 2: a regulated stablecoin can still be sent short, on the wrong network, to a look-alike address, or without a reference. Our hypothesis is that regulatory clarity will bring more conventional businesses to accept stablecoins, and that those businesses will expect the same controls they have for bank receipts. [ASSUMPTION]

4. Why existing solutions leave a gap

Clearline enters a crowded field. The fair question is what each kind of product already does.

Category	Examples	What they do well	What they leave to the business
Processors and checkout	Stripe, Coinbase Payments, HitPay	A hosted checkout. The processor receives the stablecoin and handles conversion and settlement. Stripe settles stablecoin payments to the merchant's Stripe balance in local currency; [VERIFIED][12] HitPay's Singapore merchants receive fiat from its MAS-licensed partner Triple-A. [VERIFIED][13]	A processor in the middle, with its own fees, coverage and onboarding. Stripe lists the US, with private previews in the EU, Hong Kong, Mexico and Switzerland; Singapore is not listed. [VERIFIED][12] Not designed for B2B customers who pay straight to the supplier's own wallet.
Wallet security	Safe{Wallet} with Blockaid	Warns the *sender* before they sign a transaction to a suspicious address, including look-alike addresses. [VERIFIED][14]	Protects the payer, not the payee's books. Does not say which invoice a receipt pays.
Crypto accounting and sub-ledgers	Bitwave, Request Finance, TRES, Cryptio	Mature tools that sync on-chain activity to ERPs. Bitwave supports stablecoin invoices and accounts receivable and syncs with NetSuite, Xero, QuickBooks and Sage; [VERIFIED][15] Request Finance syncs to QuickBooks Online, Xero and NetSuite; [VERIFIED][16] TRES lists native NetSuite, QuickBooks and Xero integrations; [VERIFIED][17] Cryptio integrates with NetSuite. [VERIFIED][18]	These are real competitors. "Nobody connects stablecoins to accounting" is false. Our working view, not yet tested against their products, is that they are built for crypto-native finance teams with many wallets and assets, and start after the money arrives. [ASSUMPTION]

The platforms are integrating vertically. Bessemer writes that Stripe, Circle and Coinbase are "leveraging their distribution to become one-stop shops", and that the most promising early-stage startups

[11] Gibson Dunn, "Singapore Publishes Draft Legislation to Implement Its Stablecoin Framework", September 2026. https://www.gibsondunn.com/singapore-publishes-draft-legislation-to-implement-its-stablecoin-framework/

[12] Stripe Docs, "Stablecoin payments". https://docs.stripe.com/crypto/stablecoins

[13] HitPay, "Accept Crypto". https://hitpayapp.com/crypto-payment-gateway-hitpay

[14] Safe, "Safe{Wallet} gets even safer: introducing Blockaid integration", 16 September 2024. https://safe.global/blog/safe-wallet-gets-even-safer-introducing-blockaid-integration

[15] Bitwave, "Stablecoin B2B payments". https://www.bitwave.io/request-a-demo-stablecoin-b2b-payments

[16] Request Finance Help Center, "Accounting integrations overview". https://help.request.finance/en/articles/13038304-accounting-integrations-overview

[17] TRES Finance, "Crypto accounting". https://tres.finance/lpp/crypto-accounting

[18] Cryptio, "Cryptio: the obvious choice for NetSuite users", 15 January 2026. https://blog.cryptio.co/cryptio-the-obvious-choice-for-netsuite-users

"will navigate outside the paths of Stripe, Circle, and Coinbase". [VERIFIED][1]

Where Clearline sits. Clearline is for the business whose customers pay *directly* into its own wallet, with no processor in the middle. That business keeps custody and pays no processing spread, but inherits all six failure modes in section 2.2. Our hypothesis about the wedge is specific: [ASSUMPTION]

1. **It starts before the payment.** Each invoice gets a precise instruction (exact amount, the issuer's token contract, the network and the address), and the payer can check the address they are about to use. Prevention is cheaper than investigation.
2. **It is built around exceptions.** The happy path is easy for everyone. The value is in diagnosing the short, counterfeit, wrong-network, unreferenced and duplicate payments, with evidence a person can verify.
3. **It refuses to book what it cannot verify.** Counterfeit, failed and wrong-network transfers never post. Nothing posts without a person's approval.
4. **It is neutral and read-only.** It holds no funds and favours no issuer, chain or processor.

Multi-chain coverage is a design goal, not a current fact: payment requests work on Ethereum only today (section 5.2). Whether this wedge is valuable enough to pay for is the main open question (section 10).

5. Design: how Clearline works

5.1 Principles

- **Read-only on-chain.** Clearline has no wallet, no private keys and no signing code. It reads public data. [VERIFIED: src/lib/chain/]
- **Deterministic first.** Every verdict comes from explicit rules over explicit evidence. AI may word an explanation; it never decides one.
- **Evidence with every verdict.** Each check stores what it found, where it found it, and which rule version decided. [VERIFIED: src/lib/requests/types.ts]
- **A person approves anything that changes the books.** [VERIFIED: src/lib/accounting/engine.ts]
- **Never overwrite history.** Decisions, activity and checks are appended; a person's choice never erases what the system concluded earlier. [VERIFIED: src/lib/requests/lifecycle.ts]
- **Derive state; don't store it.** A request's status is recomputed from its records every time, so it cannot drift from them. [VERIFIED: src/lib/requests/types.ts]

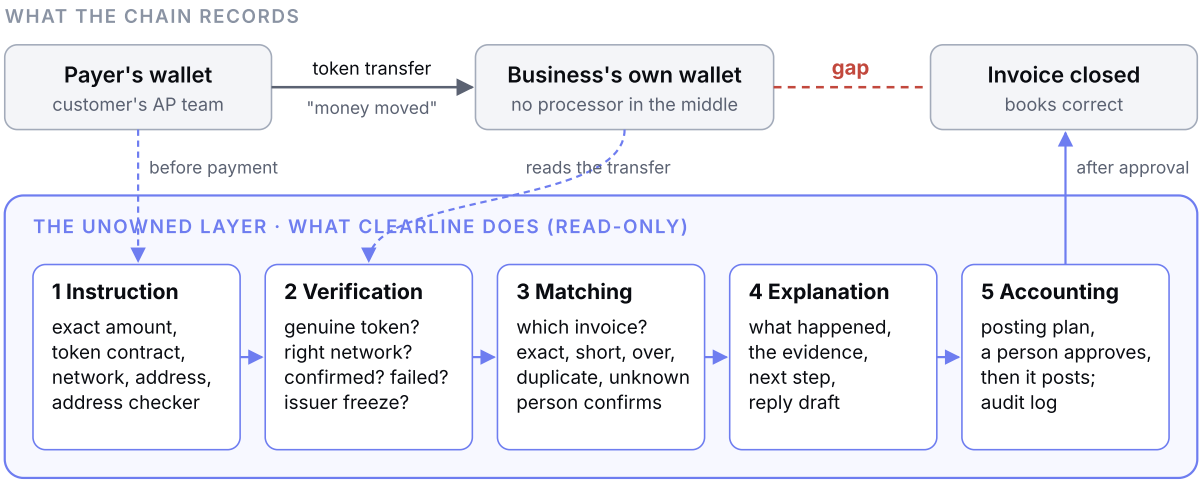


Figure 1: Where Clearline sits. The chain records the transfer; Clearline supplies the layer between "money moved" and "invoice closed".

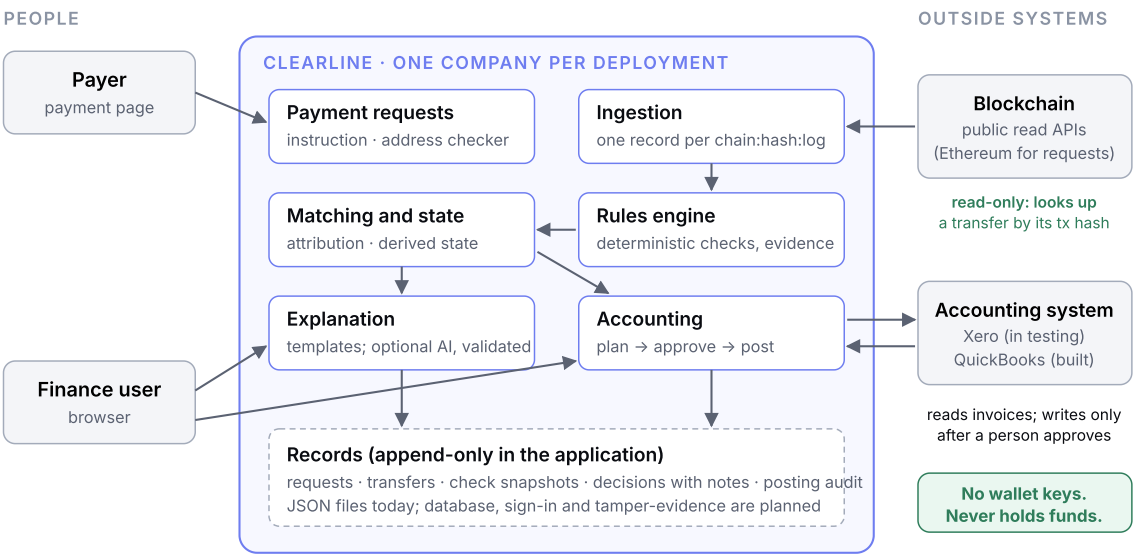


Figure 2: Architecture today. One deployment serves one company. Clearline reads the chain and writes to the books only after approval.

5.2 Payment requests: the instruction

A payment request is created for one invoice, either by hand or by importing an open invoice from the accounting system. It fixes, at creation: [VERIFIED: src/lib/requests/lifecycle.ts]

- the **amount** as an exact decimal string (never a floating-point number), at most six decimal places, which is the precision of USDC and USDT on Ethereum;
- the **token**, USDC or USDT, and the **issuer's official contract** for it;
- the **network**, Ethereum;
- the **receiving address**, which is the business's own wallet. Clearline never generates or holds addresses;
- a random public identifier for the payer's link, so links cannot be guessed from invoice numbers.

The payer sees a page with the amount, token, network and the address split into groups for checking, plus a QR code in the EIP-681 format, the Ethereum standard for encoding a token transfer as a URL. [VERIFIED: src/lib/requests/present.ts][19] The page also has an **address checker**: the payer pastes the address their wallet is about to use, and Clearline answers *match*, *look-alike (stop)*, *different* or *invalid*. A look-alike is an address that shares the first four or last four hex characters with the real one but differs elsewhere, which is the pattern address-poisoning relies on. [VERIFIED: src/lib/requests/lifecycle.ts] Only the verdict is stored, plus the pasted address when it is a look-alike, as evidence of an attack.

Coverage today: payment requests support USDC and USDT on **Ethereum only**. [VERIFIED: src/lib/requests/types.ts] Tron, which carries about 31% of stablecoin supply, [VERIFIED][4] is the most important gap and is next on the roadmap.

5.3 Detection and verification

How a transfer reaches Clearline. In demo mode, a simulated provider generates transfers, clearly labelled as simulated, and they flow through the same code as live ones. In live mode, **a person pastes the transaction ID (hash)**, often from the customer's email, and Clearline looks it up read-only on Ethereum. [VERIFIED: src/lib/requests/provider.ts] Clearline does **not** yet watch addresses for incoming transfers; that is the first item on the roadmap. Each transfer is stored once under the key `chain:hash:logIndex`, so a repeated delivery of the same event never creates a second payment, and confirmations never go backwards. [VERIFIED: src/lib/requests/lifecycle.ts]

The checks. Each transfer is run through a rules engine with a fixed evaluation order; the verdict

[19] Ethereum Improvement Proposals, ERC-681, "URL Format for Transaction Requests". https://ercs.ethereum.org/ERCS/erc-681

is the first rule that matches with high confidence. [VERIFIED: src/lib/catalogue.ts] For a payment request, a transfer counts only if all of these hold: [VERIFIED: src/lib/requests/lifecycle.ts]

Check	Rule	Can a person override it?
Succeeded	The transaction did not revert	No
Network	Sent on the request's network	No (a person may count it only if it landed in the business's own wallet on that network; it is still never posted automatically)
Address	Sent to the request's receiving address	Only by assigning an unidentified payment, with a note
Genuine token	Token contract equals the issuer's contract fixed on the request. A transfer whose symbol claims to be the token, from any other contract, is marked counterfeit	No
Timing	Mined after the request was created	Yes, with a note
Engine findings	For example an issuer freeze on the sender, read from Tether's or Circle's own contract	No
Confirmations	12 blocks on Ethereum (a sample setting; not yet editable)	n/a

If a chain data source does not answer, the check **pauses** and concludes nothing; a missing receipt is never read as a failure. [VERIFIED: src/lib/engine/run.ts, src/lib/chain/evm.ts] Every re-check is appended as a snapshot with its rule version, so the record shows how a verdict evolved.

5.4 Matching

Attribution, deciding which request a transfer belongs to, is deterministic and conservative: [VERIFIED: src/lib/requests/lifecycle.ts]

- Only open requests on the same network, whose receiving address is the transfer's destination, are candidates.
- One candidate: the address identifies the request.
- Several candidates sharing an address: **never decided by amount alone**. The transfer waits for a person; an exact-amount match is shown only as a hint.
- A transfer to the address of a request already paid: flagged as a possible duplicate.

For a payment with no usable reference (for example, sent to a general treasury address), Clearline suggests up to three open requests, scored out of 100 from evidence a person can check: the exact amount owed (40 points), a sender address this customer has paid from before (35), no other open request for the same amount (15), and paid after the invoice was sent (10). Scores of 80 or more are labelled *Strong*, 50 or more *Possible*. The score is not a probability, timing alone never produces a suggestion, and a person must confirm the match with a written reason. [VERIFIED: src/lib/requests/lifecycle.ts]

Each request moves through derived states: *awaiting payment*, *payment seen*, *confirming*, *matched*, *needs attention* and *paid*. Only a decision reaches *paid*: either a person's, or, where the business has switched it on, an automatic rule that marks exact, genuine, fully confirmed payments as paid. **That automatic rule is on only in demo mode today**; in live mode a person marks each payment paid. [VERIFIED: src/lib/requests/service.ts]

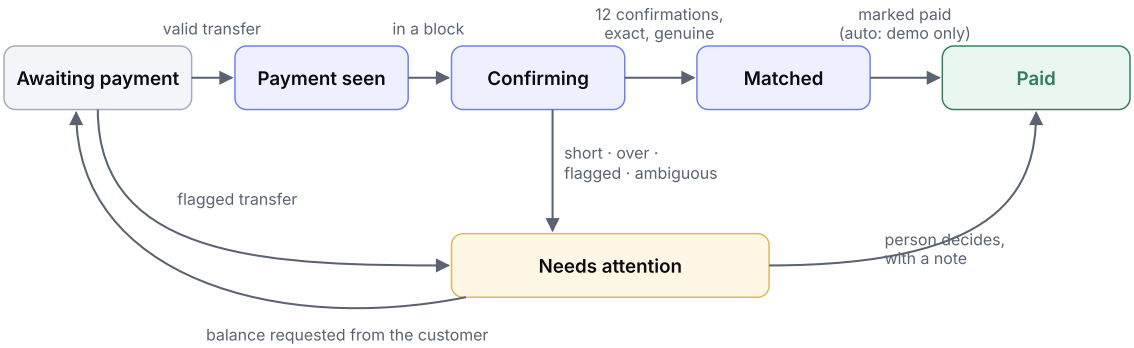


Figure 3: Request states. "Paid" is reached only by a decision: a person's, or the automatic rule (demo only today).

5.5 Explanation

For every request, Clearline writes what happened, the likely cause, the one next action, and a reply to the customer, built from the records by templates. [VERIFIED: src/lib/requests/present.ts] Two rules govern this text: a **likely cause** is labelled as an inference, not a fact (for example, "a 1.0% shortfall from an exchange's withdrawal wallet: likely fees deducted"), and replies state only facts from the records and make no promise of timing or credit.

There is also an **optional AI explanation** path on the transaction-investigation pages. A language model (Claude) writes a short paragraph about a verdict the rules have already decided. Its output is rejected, and the template used instead, if any sentence fails to cite a real evidence ID, or if it contains a number, address, hash, date or token name not in the evidence, or banned words such as "guaranteed", "refund" or "fraud". [VERIFIED: src/lib/ai/validate.ts] This path is **off by default and has not been tested on real data**. [VERIFIED: demo/REPORT.md]

5.6 Accounting

Clearline imports open sales invoices, turns each into a payment request, and after payment builds a **posting plan**: the exact lines that would go into the books. [VERIFIED: docs/ACCOUNTING_SYNC.md] A person reviews and approves the plan; the server refuses the approval if the plan changed since the page was loaded, so what is approved is exactly what posts. Each line posts with an idempotency key, so a retry never posts twice, and every approval, posting, failure and refusal is appended to an audit log. [VERIFIED: src/lib/accounting/engine.ts]

Case	What posts	Invoice after
Exact	One payment, with tx hash, token, network and USD value in the reference	Paid
Short	A partial payment and a note	Open for the shortfall
Short, accepted as settled	The payment plus a separate fee write-off line	Paid
Over, or paid twice	Nothing until a person decides; then the invoice amount once, and the excess as a separate overpayment line (refund or credit)	Paid
Same transfer reported twice	Nothing new	Unchanged
Counterfeit, failed or wrong-network	**Never posted**	Unchanged
No reference	Nothing until a person confirms the invoice	
Non-USD invoice, or token more than 0.5% off its peg	Blocked; post by hand	

USDC and USDT are valued at par, and every line says so ("no price feed connected"); a price

source can be plugged in, and a reading more than 0.5% from par stops automatic posting. Payment lines are rounded *down* to the cent, so the books never show more received than arrived. [VERIFIED: src/lib/accounting/rules.ts] All rules in this table are covered by automated tests. [VERIFIED: tests/accounting.test.ts]

Status of each accounting system. *Xero:* built and tested in automated tests against recorded API shapes; it has **not yet run against a live Xero organisation**, and the test with Xero's Demo Company is next. *QuickBooks Online:* built against the documented API shapes, not yet run on a sandbox. *NetSuite:* interface designed only. [VERIFIED: docs/ACCOUNTING_SYNC.md] The accurate summary is: **Clearline prepares the entries and, after a person approves, posts them (Xero connection in testing).**

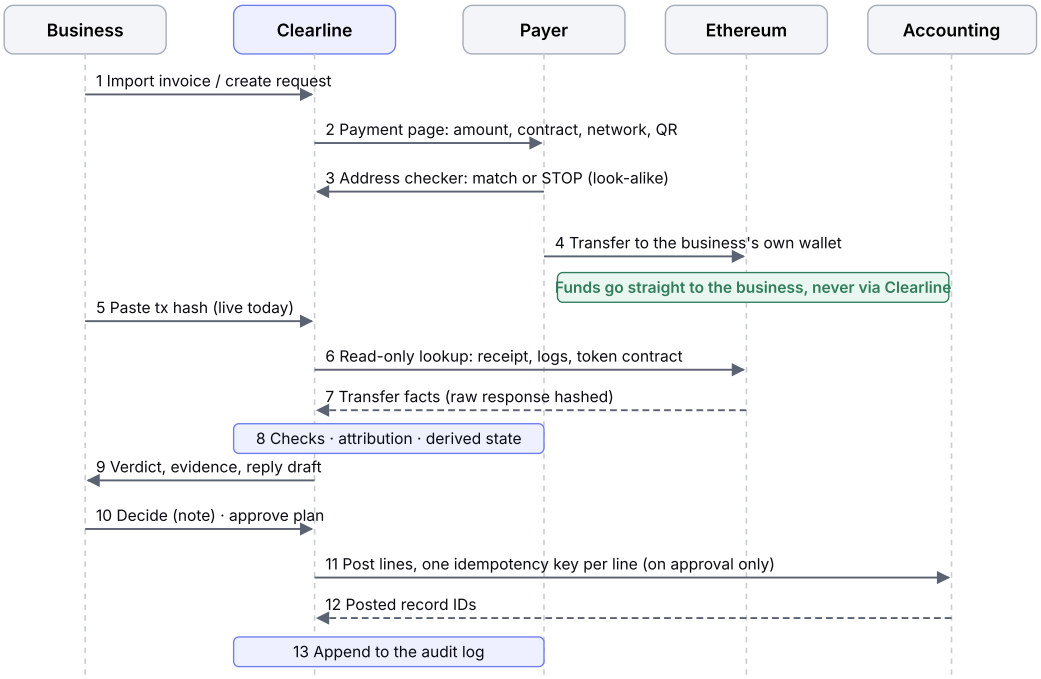


Figure 4: One payment, end to end. In live mode today, step 5 is a person pasting the transaction ID; in the demo it is automatic.

5.7 Data model

Every state shown to a user is derived from five append-only record types: the request, the observed transfer, the check snapshot, the decision (with its required note) and the posting audit entry.

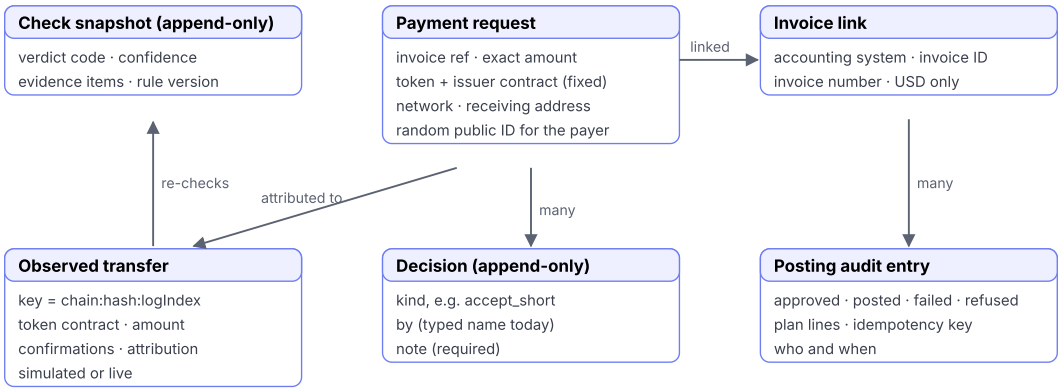


Figure 5: Core records. Decisions, activity, check snapshots and audit entries are only ever appended.

6. Trust and security model

6.1 What Clearline can touch

Area	Access
Blockchain	**Read-only.** Looks up transactions, token metadata and issuer freeze lists through public or keyed read APIs. There is no code path that can move funds. [VERIFIED: src/lib/chain/]
Funds	**None.** The receiving address is the business's own; Clearline only reads it.
Accounting system	Reads open sales invoices. Writes **only** when a person approves a plan in the *ready* state. [VERIFIED: src/lib/accounting/engine.ts] The Xero scopes requested are the ones needed to read contacts and write transactions; this is the minimum for posting payments, not read-only access. [VERIFIED: src/lib/accounting/providers/xero-auth.ts]

Why read-only matters. If Clearline received, held or forwarded customer funds, it would become a third party in the payment and, in Singapore, would likely need a Payment Services Act licence for digital payment token services. [ASSUMPTION] The market already shows the boundary: HitPay states that its Singapore stablecoin payments are processed by Triple-A, which is licensed by MAS for digital payment token services, and that HitPay itself does not handle, convert or hold stablecoins. [VERIFIED][13] Staying read-only is a design choice and a regulatory position. We have not taken legal advice on it yet, and will before the first paying customer.

6.2 Threat model

Threat	Mitigation today	Residual risk
Counterfeit tokens	Token contract must equal the issuer's, fixed at request creation; counterfeits cannot be marked paid and are never posted [VERIFIED: src/lib/requests/lifecycle.ts]	Depends on the official contract list being correct
Poisoned (look-alike) addresses	Address checker on the payment page; look-alike diagnosis on investigations [VERIFIED: src/lib/requests/lifecycle.ts]	Works only if the payer uses the checker
Impersonation of the business (a fake payment page or invoice)	Random payment-link IDs	**Not solved.** A scammer can still send a convincing fake page. Signed instructions (section 7) are the planned answer
Replay and duplicates	One record per `chain:hash:logIndex`; one idempotency key per posting line; approval tied to the exact plan [VERIFIED: tests/accounting.test.ts]	Low
Insider error	A written reason for every decision; nothing posts without approval; append-only history	**High until sign-in exists**: the actor is a typed name
Data-source failure	Checks pause and conclude nothing; a missing receipt is never read as a failure [VERIFIED: demo/REPORT.md]	Free public data sources are fragile; paid ones are planned
Server compromise	Secrets in environment variables, not in the code	**High:** see known gaps

6.3 Known gaps, and the planned fix

These are the gaps a security reviewer would find first. We list them because a pilot customer should know them before agreeing to one. [VERIFIED: docs/enterprise/SECURITY_OVERVIEW.md (PR 30)]

Gap today	Planned fix
No sign-in. The user types their name; the only gate is an optional shared password (HTTP Basic authentication)	Sign-in with email and a one-time code, then SSO; roles
Xero tokens stored unencrypted on the server's disk	Encrypt with a key from the host's secret manager; restrict file access; add a "Disconnect" that revokes the token
Audit log is append-only in the application, not tamper-evident: anyone with server access could edit it	Hash-chained entries, exported daily
Data in JSON files, one process	A managed, encrypted database in the customer's chosen region
Deletion is manual	Per-record and whole-workspace deletion, logged. Until then, pilot data is deleted by hand within 10 business days of a request
No penetration test or SOC 2 report	An independent penetration test before the first customer with real volume; SOC 2 only when customers require it

6.4 Data retention

Clearline stores invoice references, customer references, amounts, addresses, transaction records and the names typed by users. It stores no private keys and no identity documents. There is no automatic retention schedule yet; pilot data is deleted on request, by hand, and confirmed in writing. [VERIFIED: docs/enterprise/SECURITY_OVERVIEW.md (PR 30)] Each pilot can run in a dedicated deployment, including in the customer's own cloud account.

7. Toward an open standard

The deepest problem in section 2 is that a payment instruction and a payment receipt are not linked by anything both sides can verify. The invoice says one thing in a PDF; the chain records another; the payer's wallet knows neither. EIP-681 already defines how to encode a token transfer as a URL, and Clearline uses it for QR codes. [VERIFIED: src/lib/requests/present.ts] What is missing is a **signed instruction** the payer's wallet can verify, and a **receipt** the payee's auditor can verify.

We propose a small, open format with two objects (draft; not built):

```
{
  "instruction": {
    "version": "0.1",
    "issuer": { "name": "Northwind Pay Pte Ltd", "domain": "northwind.example" },
    "invoice": "INV-1042",
    "amount": { "value": "5000000000", "decimals": 6 },
    "asset": { "chain": "eip155:1", "contract": "0xA0b8...eB48", "symbol": "USDC" },
    "payTo": "0x...",
    "expires": "2026-11-30T00:00:00Z",
    "nonce": "f3b1...",
    "signature": "signed with a key published at the issuer's domain"
  },
  "receipt": {
    "instructionHash": "sha256 of the instruction",
    "transfer": { "chain": "eip155:1", "tx": "0x...", "logIndex": 7, "block": 21480123 },
    "checks": [ { "rule": "genuine_token", "result": "pass", "ruleVersion": "2026-10-22.5" } ],
    "verdict": "exact",
    "evidence": [ { "source": "rpc", "sha256": "raw response hash" } ],
```

```
    "booked": { "system": "xero", "reference": "payment id" }
  }
}
```

(Sample data. The contract shown is abbreviated; the business is the fictional one used in Clearline's demo.)

What each side gains. A wallet that reads the instruction can check the signature against the business's domain, refuse an address that does not match, and warn on a look-alike, which addresses impersonation and poisoning at the moment of payment. An accountant or auditor who receives the receipt can re-run the checks from public data and the stored response hashes, without trusting Clearline.

Why openness matters. The format is only useful if wallets read it, and wallets will read it only if many businesses issue it. A proprietary format would be a feature; an open one can become infrastructure. Each wallet that supports it makes every issuing business safer, and each issuing business makes wallet support more valuable. That two-sided effect is the long-term reason to build Clearline as a neutral layer rather than another checkout.

Honest limits. Clearline already keeps most of the receipt's ingredients internally (requests, transfer keys, check snapshots with rule versions, and SHA-256 hashes of raw chain responses on investigation pages). [VERIFIED: src/lib/view.ts] The signed instruction does not exist yet. Adoption is the hard part: payment-URL standards like EIP-681 exist, and Clearline's own payment page tells payers that not every wallet reads the amount or token from a QR code. [VERIFIED: src/app/pay/[publicId]/page.tsx] We would publish the draft, seek comment from wallet teams, and only push it once real customers use the instruction.

8. Evidence so far

What is built and tested.

Evidence	Result
Real public transactions, checked by the transaction-investigation engine against answers proven by sources outside the product (explorers, a second data provider, issuers' own blacklist events)	**59 of 59** correct after fixes (43 of 59 before), and **52 of 52** on a held-out set built afterwards; 0 false alarms on 12 normal payments; 0 wrong answers with high confidence, down from 13 [VERIFIED: demo/REPORT.md]
Coverage of those cases	Tron, Ethereum, BNB Chain, Polygon, Arbitrum and Base: failed transfers 21/21, issuer freezes 11/11, counterfeit USDT 13/13, wrong network 6/6, address poisoning 2/2 [VERIFIED: demo/REPORT.md]
Automated tests	**573 passing** (57 network re-recording tests skipped), run on 29 October 2026, including 23 posting-rule and Xero/QuickBooks request-shape tests and 62 replays of recorded real cases [VERIFIED: npm test on qa-polish (PR 31), run 29 Oct 2026]
End-to-end demo	113 of 113 browser checks pass, three runs in a row; all seven payment situations (routine, short, counterfeit, wrong network, unidentified, look-alike, duplicate) play through to the books [VERIFIED: demo/QA_REPORT.md (PR 31)]

What is not yet tested or built. The real-data test covered the investigation engine, not payment-request matching: matching has been tested only on simulated data. The Xero adapter has not run on a live organisation; QuickBooks has not run on a sandbox. The AI explanation has not been tested on real data. Address watching, Tron payment requests, a month-end report, sign-in and a database do not exist yet. All accuracy figures come from tests the builder designed; an independent reviewer re-checked 7 cases on-chain, and all matched. [VERIFIED: demo/REPORT.md] A small sample with known answers shows the rules work as designed. It does not show accuracy on a real business's mix of

payments.

Customers. None. No customer calls have been held. The first outreach emails were sent on 29 October 2026. The founding pilot offer and its price are untested hypotheses. [ASSUMPTION]

9. Business model and path

Who pays first. Our hypothesis is that the first buyer is the founder or finance lead of a small or mid-sized business in Singapore or Hong Kong that invoices in USD and is paid in USDC or USDT directly to its own wallet: Web3 service firms, agencies, exporters and B2B software companies. [ASSUMPTION] They feel the pain in exceptions and month-end, and they cannot use a processor or do not want one. This must be confirmed in customer calls before anything else is built.

Pricing hypothesis. A founding pilot at US$300 for 30 days, then US$200 a month. [ASSUMPTION] The pilot is deliberately paid: a business that pays has the problem. Both numbers are placeholders to be tested, not a validated price. Longer term, two models are possible: a subscription per finance team priced by payment volume, or a per-exception fee embedded in payment companies' products. [GUESS] We do not yet know which customers will prefer.

The path: wedge, network, standard.

1. **Wedge (now):** exception handling and booking for businesses paid directly. Success is measured in hours saved and errors caught per month, in pilots.
2. **Network (later):** every payment page teaches a payer to verify before sending. Repeated payments build a history of known sender addresses per customer, which makes matching better for every business on the network. [ASSUMPTION]
3. **Standard (much later):** publish the instruction and receipt format, and work with wallets to read it (section 7).

What would change the plan. If calls show that exceptions are rare or cheap, the wedge is wrong. If the main pain is on Tron, Tron comes first. If buyers want a processor, Clearline is the wrong product for them and we should say so.

Risk	Why it matters	What we would do

10. Risks and open questions

Risk	Why it matters	What we would do
Checkout tools absorb the problem	If most businesses move to processors, direct payments shrink	Stay neutral and focus on businesses that keep custody; make the receipt format useful to processors too
Crypto accounting tools add exception handling	Bitwave, Request Finance and others already have ERP sync and customers	Compete on prevention (the instruction) and on depth of exception handling; test this claim against their products early
Licensing boundary	Any feature that touches funds (refunds, conversion, sweeping) could make Clearline a licensable intermediary	Keep the “never holds funds” line; take legal advice before the first paying customer
Data access	Free public chain data is fragile; detection without a pasted hash needs address watching and paid data providers	Paid providers with a fallback per chain; budget for them in pricing
Accuracy limits	Rules tested on a small sample; matching untested on real payments	Measure on each pilot’s real cases and publish the error rate
Security maturity	No sign-in, unencrypted tokens; a security review would stop a pilot	Fix the known gaps (section 6.3) before the first live pilot
Single founder	Execution and continuity risk	Narrow scope; hire or partner for security and sales once there is revenue
Demand	Zero customers; the problem’s cost is not measured	The first 20 customer calls are the most important work

Open questions we cannot answer yet. How many exceptions does a typical business see per month? Who owns the problem: finance, operations or the founder? Will businesses connect their books to a young product? Is Ethereum-first right, given Tron’s share of supply? Is US$200 a month too much for a small team, or too little to build a business on?

11. Roadmap (12 to 24 months)

Each phase begins only when the evidence from the one before supports it.

When	What	Gate to move on
Months 0–3	Xero Demo Company live test. Fix the security gaps for pilots (sign-in, encrypted tokens). Address watching on Ethereum, so correct payments are detected without a pasted hash. First paid pilots	3 paying pilots; exceptions measured on real data
Months 3–6	Payment requests on Tron. QuickBooks sandbox and first live run. Editable settings (confirmations, accepted tokens). Tamper-evident audit log. Database. A month-end report	Pilots renew; matching accuracy published from real cases
Months 6–12	More networks (BNB Chain, Base, Polygon, Arbitrum). Non-USD invoices with a price source. Independent penetration test. Publish the draft instruction and receipt format	20 paying customers; one wallet team engaged on the format

When	What	Gate to move on
Months 12–24	NetSuite. Signed instructions verified by at least one wallet. SOC 2 if customers require it	Revenue that supports a team

12. Conclusion

Stablecoins already move hundreds of billions of dollars of real commerce, and regulation is making them more ordinary. The money moves well. What does not yet work well is everything a business needs after the transfer: knowing the payment is genuine, complete, on the right network, for a specific invoice, and correctly in the books. Clearline is a narrow attempt to make that layer reliable for businesses paid straight into their own wallets: precise instructions, deterministic checks, conservative matching, plain explanations, and accounting entries that post only with a person's approval, all without ever holding funds.

The product works in demo and on real public transactions. It has no customers yet, and its accounting connection has not yet run live. The next proof is not more code: it is whether real finance teams have this problem often enough to pay to solve it.

Appendix A. Glossary

Term	Plain meaning
Stablecoin	A token designed to hold a steady value, usually US$1. USDC (Circle) and USDT (Tether) are the largest
Token contract	The program on the blockchain that defines a token. Only the issuer's contract makes a USDC or USDT genuine
Network (chain)	A blockchain such as Ethereum or Tron. The same token exists separately on several networks
Transaction ID (hash)	A unique identifier for a blockchain transaction, used to look it up
Confirmations	The number of blocks added after the one that holds a transaction. More confirmations mean it is harder to reverse
Address poisoning	A scam: the attacker sends a tiny transfer from an address that looks like a real one, hoping the victim later copies it from their history
Counterfeit token	A token that copies a real token's name, from a different contract. It has no value
Wrong-network payment	A transfer sent on a different network from the one requested
Issuer freeze	A block the issuer places on an address, so its tokens cannot move
Posting	Writing an entry into the accounting system, for example a payment against an invoice
Idempotency key	A unique label on a write, so that sending it twice has the same effect as sending it once
Read-only	Able to look, not to act. Clearline reads the blockchain and never signs or sends a transaction
EIP-681	An Ethereum standard for writing a payment request as a URL, used in QR codes

Case	What arrived	What Clearline concludes	What it does

Appendix B. Worked examples

All examples use Clearline's demo: a **fictional** company, Northwind Pay Pte Ltd, its fictional customers, and simulated transfers. They run through the same code as live transfers. [VERIFIED: src/lib/requests/demo.ts]

Case	What arrived	What Clearline concludes	What it does
a. Routine (Acme Logistics)	Exact amount of genuine USDC to the invoice's address	Matched; 12 confirmations	Marked paid by the demo's automatic rule; one payment line posts on approval
b. Short (Brightline Studio)	4,950.00 of 5,000.00 USDC, from an exchange's withdrawal wallet	Short by 50.00 USDC (1.0%). Likely cause, labelled an inference: fees deducted by the exchange	Drafts a polite request for the balance; partial payment and note, or payment plus fee write-off if accepted as settled
c. Counterfeit (Orbit Retail)	7,500 "USDT" from a contract Tether does not control	Not genuine USDT	Cannot be marked paid; never posted; drafts a non-accusing reply
d. Wrong network (Meridian Travel)	3,200 USDT on Tron, to the business's old Tron wallet, for an Ethereum request	Wrong network, but in the business's own wallet	Treasury confirms, then a person may count it; never posted automatically
e. Unidentified (Kite Analytics)	2,180.00 USDC to the main address, no reference	Suggests INV-1045: exact amount (40), known sender (35), unique amount (15), after the invoice (10) = 100, *Strong*	A person confirms with a reason; then it posts as exact
f. Look-alike (Fernhill Foods)	Nothing yet: the payer pastes an address from their wallet history	Look-alike: same first and last four characters	"Stop: don't send to this address", before any money moves
g. Duplicate (Kestrel Freight)	1,980.00 USDC twice, four minutes apart	Paid twice	Drafts a refund instruction for treasury and a reply; posts one payment and one overpayment marked for refund

Example posting plan (case g, after the refund decision). Two lines post, each with its own idempotency key: [VERIFIED: tests/accounting.test.ts]

Line	Amount (USD)	Account	Reference
Payment	1,980.00	Wallet (bank) account	1,980.00 USDC on Ethereum, tx hash, valued at par (no price feed connected)
Overpayment	1,980.00	Wallet (bank) account	Overpayment to refund: the same invoice paid twice

Appendix C. References

1. Bessemer Venture Partners, C. Birnbaum, E. Kaplan and B. Nydick, "Stablecoins: from DeFi primitive to global financial infrastructure", 22 April 2026. https://www.bvp.com/atlas/stablecoins-from-defi-primitive-to-global-financial-infrastructure
2. Reap, "Stablecoin Statistics & Data 2026: All You Need To Know", 14 August 2026, citing BIS, BCG × Allium, McKinsey, DefiLlama and EY-Parthenon. https://reap.global/blog/stablecoin-statistics-2026
3. Cointelegraph, "Address poisoning: copy-paste mistake leads to $50M USDT loss", 20 December 2025. https://cointelegraph.com/news/address-poisoning-copy-paste-mistake-50m-usdt-loss
4. Etherscan, transaction 0xc0514a79...6a0f. https://etherscan.io/tx/0xc0514a795f065fce8a3e1238a1ba480e8a792d45f6431ee4ba8d800bedd86a0f
5. U.S. Government Publishing Office, Public Law 119-27, "An act to provide for the regulation of payment stablecoins", 18 July 2025. https://www.govinfo.gov/metadata/pkg/PLAW-119publ27/mods.xml
6. Decrypt, "OCC finalizes GENIUS Act stablecoin rules ahead of January deadline", 21 October 2026. https://decrypt.co/381502/occ-finalizes-genius-act-stablecoin-rules-ahead-of-january-deadline
7. European Banking Authority, "Asset-referenced and e-money tokens (MiCA)". https://eba.europa.eu/regulation-and-policy/asset-referenced-and-e-money-tokens-mica
8. FSMA (Belgium), "Crypto-asset service providers: end of the transitional period", 25 June 2026. https://www.fsma.be/en/news/crypto-assets-service-providers-end-transitional-period
9. Hong Kong Government News, "Stablecoins laws effective Aug 1", 6 June 2025. https://www.news.gov.hk/eng/2025/06/20250606/20250606_140750_137.html
10. Charltons, "HKMA Issues First Stablecoin Issuer Licences", April 2026. https://www.charltonslaw.com/hkma-issues-first-stablecoin-issuer-licences/
11. Gibson Dunn, "Singapore Publishes Draft Legislation to Implement Its Stablecoin Framework", September 2026. https://www.gibsondunn.com/singapore-publishes-draft-legislation-to-implement-its-stablecoin-framework/
12. Stripe Docs, "Stablecoin payments". https://docs.stripe.com/crypto/stablecoins
13. HitPay, "Accept Crypto". https://hitpayapp.com/crypto-payment-gateway-hitpay
14. Safe, "Safe{Wallet} gets even safer: introducing Blockaid integration", 16 September 2024. https://safe.global/blog/safe-wallet-gets-even-safer-introducing-blockaid-integration
15. Bitwave, "Stablecoin B2B payments". https://www.bitwave.io/request-a-demo-stablecoin-b2b-payments
16. Request Finance Help Center, "Accounting integrations overview". https://help.request.finance/en/articles/13038304-accounting-integrations-overview
17. TRES Finance, "Crypto accounting". https://tres.finance/lpp/crypto-accounting
18. Cryptio, "Cryptio: the obvious choice for NetSuite users", 15 January 2026. https://blog.cryptio.co/cryptio-the-obvious-choice-for-netsuite-users
19. Ethereum Improvement Proposals, ERC-681, "URL Format for Transaction Requests". https://ercs.ethereum.org/ERCS/erc-681

All web sources were opened on 29 October 2026. Product claims are mapped to files and tests in the accompanying claims audit.